

Learning Objectives

Learners will be able to...

- Use basic regular expression functions and symbols
- Extract encoded text from files using unicode
- Chunk and chink text
- Apply NER to text
- Compare segmentation to tokenization and use the `RegexpTokenizer()` method, and manually segment text

info

Make Sure to Know

Intermediate Python

Limitations

In this assignment, we only work with the NLTK library.

Use Regex

Definition: Regular Expression

We are now ready to start parsing text. One of the most common and powerful ways of parsing text is with regular expressions.

- Regular expressions, *regex* or *regexp* for short, can be used to locate and replace patterns in a string of text.
- Python is not the only language that can use regex (for example, R and Java can use regex as well).
- Regex can be in the form of either meta characters or literal characters.
 - Literal characters, or *literals* for short, represent basic characters that can be used to represent different meanings, such as: 'a', 'b', 'c', '1', '2', '3', and so forth.
 - Meta characters, on the other hand, represent characters with a special and constant meaning, such as: '\$', '*', '+', and so forth.

Regular expressions are commonly used for:

- *Search and match* patterns in a string.
- *Split* strings with a particular pattern into substrings with a particular pattern.
- *Findall* strings with a particular pattern.
- *Sub* out particular patterns in a string.

You must import the `re` module before using regular expressions. Three commonly-used regex functions are `re.search`, `re.findall`, and `re.match`.

▼ `re.search()` vs. `re.findall()` vs. `re.match()`

- The `re.search` function stops searching for the pattern once it finds a match in a string. It will find the first occurrence of the pattern(s) in each string.
- The `re.findall` module searches for all occurrences of the pattern in a string.
- The `re.match` function searches for the first occurrence of the pattern, just as the `re.search` function does. However, if a string is broken up by line and there is a first occurrence of the match on any line other than the first, the `re.match` function will not find it. This is because `re.match` only searches for the pattern(s) on the first line of each string.

Use `re.search`

Import the applicable processes from the NLTK library used for applying regex in these exercises.
We need to import the re module.

```
import re
from re import finditer
```

For this exercise, we will analyze the string contained in the variable called `adv_sentence`.

```
adv_sentence = "After last summer's adventures, we're up to any task!"
```

From here, we can use the `re.search()` function to *search* for patterns in the string. Remember, this function will search for the pattern then return the first occurrence of the pattern found in the string.

In the code shown below, we tell the computer what pattern we are looking for in our string. We type the character `r` right before we type our pattern. Here, we have a string pattern, looking for all instances of the string `'adventure'`.

```
pattern = r'...adventure'
```

Using a conditional, we have the ability to search for matches. We first tell the computer what constitutes a match using the variable we created called `match`, along with `re.search`. If there is a match, the `start()` method is used to tell the user at which character in the string the matching substring begins.

Enter the below code, then select the TRY IT button to run the file.

```
match = re.search(pattern, adv_sentence)
if match != None:
    print(f"\nThere is a match. The string 'adventure' starts on the {match.start()}th letter.")
else:
    print("\nThe computer wasn't able to find a match.")
```

Your output should display as shown below.

```
There is a match. The string 'adventure' starts on the 17th letter.
```

▼ Did you notice?

The computer told us that the string 'adventure' was found in the string called `adv_sentence`. Consider the following scenario.

Assume we change the value of `adv_sentence` so only the string 'adventure' was included, and if we also change the value of `pattern` so it included the plural string 'adventures', as shown below.

```
adv_sentence = "After last summer's adventure, we're up to any  
task!"  
pattern = r'...adventures'
```

Then, the output to our conditional would display as shown below:

```
The computer wasn't able to find a match.
```

'adventure' is a substring of 'adventures', but 'adventures' is not a substring of 'adventure'.

challenge

Try this variation:

We can also search for multiple string patterns at once by inserting each string value we want to search for in a list, as shown below.

Enter the below code, then select the TRY IT button to run the file.

```
pattern2 = ['adventures', 'adventure', 'dog']
for element in pattern2:
    if re.search(element, adv_sentence):
        print(f'\nMatch! The computer found the pattern "{element}" in the sentence: "{adv_sentence}"')
    else:
        print(f'\nNo Match! The computer was not able to find the pattern "{element}" in the sentence: "{adv_sentence}"')
```

Your output should display as shown below.

```
Match! The computer found the pattern "adventures" in the
sentence: "After last summer's adventures, we're up to any
task!"
Match! The computer found the pattern "adventure" in the
sentence: "After last summer's adventures, we're up to any
task!"
No Match! The computer was not able to find the pattern
"dog" in the sentence: "After last summer's adventures,
we're up to any task!"
```

Use re.findall

In this next exercise, we will work with the list called `art_sentences`, where each element in the list represents a sentence.

```
art_sentences = ['There is no greater joy for an artist than to
know their work is in the hands of someone who truly
understands it.', 'These guys have state of the art
equipment.']
```

From here, we can use the `re.findall` module to search for all occurrences of patterns in the string. Shown below is the command we can use to tell the computer what patterns we are looking for in our strings.

```
pattern3 = ['art', 'artist', 'dog']
```

In the code shown below, we use `re.findall`, along with a nested loop to find all occurrences of the patterns in the example strings contained in the list.

Enter the below code, then select the TRY IT button to run the file.

```
for art_sentence in art_sentences:
    for each_pattern3 in pattern3:
        if re.findall(each_pattern3, art_sentence):
            print(f'\nMatch! The pattern "{each_pattern3}" is in the
                sentence: "{art_sentence}"')
        else:
            print(f'\nNo Match! The pattern "{each_pattern3}" is not
                in the sentence: "{art_sentence}"')
```

Your output should display as shown below.

```
Match! The pattern "art" is in the sentence: "There is no
greater joy for an artist than to know their work is in the
hands of someone who truly understands it."
Match! The pattern "artist" is in the sentence: "There is no
greater joy for an artist than to know their work is in the
hands of someone who truly understands it."
No Match! The pattern "dog" is not in the sentence: "There is no
greater joy for an artist than to know their work is in the
hands of someone who truly understands it."
Match! The pattern "art" is in the sentence: "These guys have
state of the art equipment."
No Match! The pattern "artist" is not in the sentence: "These
guys have state of the art equipment."
No Match! The pattern "dog" is not in the sentence: "These guys
have state of the art equipment."
```

challenge

Try this variation:

When we find a matching substring, we are going to use the `finditer` function, which returns an iterable object about the substring. The `.span()` method returns a tuple including the starting and ending indexes of the matched pattern in each string. The `.group()` method

returns the matching substring. We can add this to the loop we created in the exercise above to return the starting and ending index values (the location) of each matching substring.

Enter the below code, then select the TRY IT button to run the file.

```
for art_sentence in art_sentences:
    for each_pattern3 in pattern3:
        if re.findall(each_pattern3, art_sentence):
            for each_pattern_found in finditer(each_pattern3,
            art_sentence):
                print(f'\nMatch! Location:
                {each_pattern_found.span()}. The pattern "
                {each_pattern_found.group()}" is in the sentence: "
                {art_sentence}')
            else:
                print(f'\nNo Match! The pattern "{each_pattern3}" is
                not in the sentence: "{art_sentence}')
```

Your output should display as shown below.

Match! Location: (31, 34). The pattern "art" is in the sentence: "There is no greater joy for an artist than to know their work is in the hands of someone who truly understands it."

Match! Location: (31, 37). The pattern "artist" is in the sentence: "There is no greater joy for an artist than to know their work is in the hands of someone who truly understands it."

No Match! The pattern "dog" is not in the sentence: "There is no greater joy for an artist than to know their work is in the hands of someone who truly understands it."

Match! Location: (29, 32). The pattern "art" is in the sentence: "These guys have state of the art equipment."

No Match! The pattern "artist" is not in the sentence: "These guys have state of the art equipment."

No Match! The pattern "dog" is not in the sentence: "These guys have state of the art equipment."

▼ Before you move on.

Regex parsing is useful for searching and finding sub-patterns of the match.

- For example, regex could be used to find and match a pattern only if the pattern has a specified sub-pattern.

On the next page of this assignment, we will go over how regex parsing is used for chunking data in NLP.

Apply Regex Symbols

Adding meta characters, operators, and quantifiers to your regular expressions increases their power. You can be more precise about the kinds of patterns you want to find.

Use Basic Meta Characters, Operators, & Quantifiers

Basic meta characters are used to represent literal characters. In the table below, four basic meta characters are defined: `.`, `\w`, `\d`, and `\s`.

Meta Character	Application	Example
.	A period that can be used to represent any character except new line characters.	' <code>.i</code> ' will match for any character followed by an 'i'.
\w	A backslash followed by a lowercase 'w' is used to represent any word character. A word character includes lower- and uppercase letters a to Z, digits 0 to 9, and the underscore character '_'.	' <code>\w\wy</code> ' will match for any two word characters followed by a 'y'.
\d	A backslash followed by a lowercase 'd' is used to represent any digit.	' <code>\d0</code> ' will match for any digit followed by a zero, 0.
\s	A backslash followed by a lowercase 's' is used to represent whitespace (created by using the <i>space</i> , <i>tab</i> , or <i>enter</i> key).	' <code>\w\s\w</code> ' will match for any word character followed by a white space followed by any word character.

Start by importing the `re` module.

```
import re
```

For this exercise, we will analyze the string contained in the variable called king.

```
king = 'After the death of the king, everyone wanted to be a king.'
```

Create pattern1 to find if the characters 'king' followed by a white space occur in the string king.

```
pattern1 = r'\sking'
match1 = re.findall(pattern1, king)

if match1:
    print(f'\nMatch! The pattern "{pattern1}" is in the sentence: "{king}"')
else:
    print(f'\nNo Match! The pattern "{pattern1}" is not in the sentence: "{king}"')
```

Your output should display as shown below.

```
Match! The pattern "\sking" is in the sentence: "After the death
of the king, everyone wanted to be a king."
```

Basic operators ^ and \$ are used to identify matches of a specified character(s) at either the beginning or ending of a string, respectively.

Operator	Application
^__	A carrot symbol can be used to match any <u>start</u> of a string with the specified character(s) written to the <u>right</u> of the carrot symbol.
__\$	A dollar symbol can be used to match any <u>end</u> of a string with the specified character(s) written to the <u>left</u> of the dollar symbol.

Create pattern2 to find if the characters 'k..g' occur at the end of string king.

```

pattern2 = r'k..g$'
match2 = re.findall(pattern2, king)

if match2:
    print(f'\nMatch! The pattern {pattern2} is in the sentence: {king}')
else:
    print(f'\nNo Match! The pattern {pattern2} is not in the sentence: {king}')

```

Your output should display as shown below.

```

No Match! The pattern "k..g$" is not in the sentence: "After the
death of the king, everyone wanted to be a king."

```

▼ Why is this not a match?

The pattern we used to determine a match was as follows: `pattern2 = r'k..g$'`. Because string `king` ends with the characters `'... king.'`, and not `'... king'`, this is not a match.

Quantifiers are used to identify characters with various occurrences.

Quantifier	Application
<code>- *</code>	An asterisk symbol that can be used to match zero or more occurrences of the character(s) written to the left of the asterisk symbol along with the character written to the right of the asterisk symbol in any string.
<code>+</code>	An addition symbol that can be used to match one or more occurrences of the character(s) written to the left of the addition symbol along with the character written to the right of the addition symbol in any string.
<code>_?_</code>	A question mark symbol that can be used to match zero or one occurrence of the character(s) written to the left of the question mark symbol along with the character written to the right of the question mark symbol in any string.
	A pair of curly braces that can be used to match a specified number of

`_{}_`

occurrences of the character(s) written to the left of the open curly brace symbol along with the character written to the right of the closed curly brace symbol in any string. The specified number of occurrences is written enclosed within the curly braces.

▼ What are closures?

- **Closure:** Also referred to as a *Kleene closure* (in reference to the mathematician, Stephen Cole Kleene), this represents a stoppage of a specified set under the operators, * or +.
 - *: The criteria can occur 0 or more times.
 - +: The criteria must occur at least once (1 or more times).

For this part of the exercise, use string `dessert`. Create `pattern3` to find if the characters 'de' followed by any number of any character preceding characters 'ert' occurs in string `dessert`.

```
dessert = 'The most awesome meal of the day is dessert because
it can be whatever you want.'
pattern3 = r'de.*ert'
match3 = re.findall(pattern3, dessert)

if match3:
    print(f'\nMatch! The pattern "{pattern3}" is in the sentence:
    "{dessert}"')
else:
    print(f'\nNo Match! The pattern "{pattern3}" is not in the
    sentence: "{dessert}"')
```

Your output should display as shown below.

```
Match! The pattern "de.*ert" is in the sentence: "The most
awesome meal of the day is dessert because it can be whatever
you want."
```

Use Square Brackets & Parentheses

We can use square brackets `[_]` to match a specified character(s) with that in a string. Specifying either one of a set of characters or one of a range of characters, we can write the pattern. We can also negate patterns.

- *One of a set of characters:* In the example shown below, the pattern will try to match for any 1s, 2s or 3s in a specified string.

```
python    pattern4 = r'[123]'
```

- *One of a range of characters*: In the example shown below, the pattern will try to match for any 1s, 2s or 3s in a specified string.

```
python    pattern5 = r'[1-3]'
```

- *Negates one of a set or range of characters*: In the example shown below, the pattern will try to match for any characters that are not 1s, 2s or 3s in a specified string. In order to negate, we use the carrot symbol ^ at the beginning of the expression enclosed within the brackets.

```
python    pattern6 = r'^[1-3]'
```

For this part of the exercise, use string numbers. Create list patterns456 that contains pattern4, pattern5 and pattern6 as created above. Iterate through each pattern to find if..

1. pattern4: ..one or more of the numbers 1, 2 or 3 occurs in string numbers.

1. pattern5: ..one or more of the numbers 1, 2 or 3 occurs in string numbers.

1. pattern6: ..one or more of any number other than 1, 2 or 3 occurs in string numbers.

```
numbers = '315-897-4873'

patterns456 = [pattern4, pattern5, pattern6]
for pattern in patterns456:
    if re.findall(pattern, numbers):
        print(f'\nMatch! At least one of the numbers in {pattern} is
              in the string: {numbers}')
    else:
        print(f'\nNo Match! None of the numbers in {pattern} are in
              the string: {numbers}')
```

Your output should display as shown below.

```
Match! At least one of the numbers in "[123]" is in the string:
"315-897-4873"
Match! At least one of the numbers in "[1-3]" is in the string:
"315-897-4873"
Match! At least one of the numbers in "[^1-3]" is in the string:
"315-897-4873"
```

We can **capture** a group of specified characters by enclosing the specified group within parentheses (_). There are two ways we can capture text.

1. *Capture groups*: For example, we can search a specified text using a pattern that captures one or more digits separated by two hyphens, as shown in the code below.

```
python    pattern7 = r'(\d+-\d+-\d+)'
```

1. *Capture groups within groups*: In this example, we can also search a specified text for a pattern that only captures the first one or more digits

separated by the first hyphen in a pattern that captures one or more digits separated by two hyphens, as shown in the code below.

```
python      pattern8 = r'(\d+)-\d+-\d+'
```

For this part of the exercise, continue to use string numbers.

Create list patterns78 that contains pattern7 and pattern8 as created above. Iterate through each pattern to find if..

1. pattern7: ..one or more digit characters is followed by a hyphen '-' is followed by one or more digit characters is followed by a hyphen '-' is followed by one or more digit characters in string numbers.

1. pattern8: ..one or more digit characters is followed by a hyphen '-' is followed by one or more digit characters is followed by a hyphen '-' is followed by one or more digit characters in string numbers.

```
patterns78 = [pattern7, pattern8]
for pattern in patterns78:
    if re.findall(pattern, numbers):
        print(f'\nMatch! At least one of the numbers in {pattern} is
              in the string: {numbers}')
        print(f'Pattern found: {re.findall(pattern, numbers)}')
    else:
        print(f'\nNo Match! None of the numbers in {pattern} are in
              the string: {numbers}')
```

Your output should display as shown below.

```
Match! At least one of the numbers in "(\d+-\d+-\d+)" is in the
string: "315-897-4873"
Pattern found: ['315-897-4873']
Match! At least one of the numbers in "(\d+)-\d+-\d+" is in the
string: "315-897-4873"
Pattern found: ['315']
```

challenge

Try this variation:

We can use either square brackets `[_]` or the *or* symbol `|` with parentheses `(_)` to create *or* conditions.

1. Use `[_]`: In the example shown below, the pattern will try to match for any characters that are 1s, 2s or 3s which are followed by three digits in a specified string without capturing 1, 2 or 3.

```
python    pattern9 = r'[1-3]\d{3}'
```

1. Use `/ and ( )`: In the example shown below, the pattern will try to match for any characters that are 1s, 2s or 3s which are followed by three digits in a specified string, and will capture 1, 2 or 3.

```
python    pattern10 = r'(1|2|3)\d{3}'
```

For this variation, use string `numbers2`.

Create list `patterns910` that contains `pattern9` and `pattern10` as created above.

Predict what you think will happen, enter the code, then select the TRY IT button to run the file.

```
numbers2 = '14590813302'

patterns910 = [pattern9, pattern10]
for pattern in patterns910:
    if re.findall(pattern, numbers2):
        print(f'\nMatch! At least one of the numbers in
              {pattern} is in the string: {numbers2}')
        print(f'Pattern found: {re.findall(pattern, numbers2)}')
    else:
        print(f'\nNo Match! None of the numbers in {pattern} are
              in the string: {number2}')
```

Your output should display as shown below.

```
Match! At least one of the numbers in "[1-3]\d{3}" is in the
string: "14590813302"
Pattern found: ['1459', '1330']
Match! At least one of the numbers in "(1|2|3)\d{3}" is in
the string: "14590813302"
Pattern found: ['1', '1']
```

Utilize Unicode

Definition: Unicode

So far, we have been searching strings with English words. But NLP is not limited to just the English language. We can work with other languages, but how a text is encoded can help or hinder this process.

- Unicode defines every character in every spoken language of the world by assigning it a unique number. The Unicode standard is maintained by the Unicode Consortium and defines over 1.4 million characters from more than 150 modern and historic scripts. It even includes emojis.
- Each assigned number to each character is called a *code point*. In Python, we use `\uNNNN` as syntax to represent the form of each code point, where `NNNN` represents the code point in hexadecimal.
- Unicode is encoded as a stream of bytes; this is because each character cannot always be packed into one single byte of data.
 - ASCII and Latin-2 encodings cannot support the entire sphere of Unicode at once because they each work by packing one code point per one single byte of data.
 - Other encodings, such as UTF-8, do allow for one code point to use more than one byte of data.
- *Decoding*: when we translate text to Unicode.
- *Encoding*: when we translate Unicode to text.

Use Latin-2 Encoding

The path to the file called `unicode_text.txt` is assigned to the variable called `path_1`.

```
path_1 = 'exercises/unicode_text.txt'
```

In the code shown below, we use the variable called `path_1` along with the `open` function to assign the text from the file to the variable called `text_latin2`.

- We can return the text from the file using Latin-2 encoding by iterating through each line with a `for` loop along with the `strip` method.

▼ Explained: `strip()`

Without any arguments, the `strip()` method removes all leading and trailing characters from the string.


```
string_name.strip()
```

We can also use the `strip` method to remove specified characters if they are leading or trailing characters in a string.

```
string_name.strip(characters_to_remove)
```

Enter the below code, then select the TRY IT button to run the file.

```
with open(path_1, encoding = 'latin2') as text_latin2:
    print('\nPlain Text from path_1 using Latin-2 encoding:')
    for line in text_latin2:
        line = line.strip()
        print(line)
```

Your output should display as shown below. Notice how the Latin-2 encoding has trouble with non-ASCII characters.

```
Plain Text from path_1 using Latin-2 encoding:
Asynchrones Schreiben erh  st die Leistung ein wenig, es
verringert jedoch die Zuverl  ssigkeit, da die Gefahr eines
Datenverlusts besteht, falls der Server zwischen der
Annahmebest  tigung des Schreibauftrags und dem tats  chlichen
Schreibvorgang auf der Festplatte abst  rzt.
Aus diesem Grund erhalten Sie f  r jede bei uns get  tigte
Bestellung eine Annahmebest  tigung, die noch einmal alle
wichtigen Daten des Auftrags zusammenfa  ssen.
Durch Vorlage der Annahmebest  tigung Ihrer Fakult  t und des
ausgef  llten Antrags auf Registrierung im
Studierendensekretariat (deutsche Promovierende) oder beim
International Admissions und Services (IAS) (internationale
Promovierende) werden Sie als Doktorand*in registriert.
```

▼ Explained: ASCII

American Standard Code for Information Interchange, or ASCII, is a character-encoding standard that represents each English character with a number from 0 to 127.

Importing the `string` module allows us to access constants defined in `ascii_letters` and `printable`.

Enter the below code, then select the TRY IT button to run the file.

```
import string
print(f'\nASCII Letters: {string.ascii_letters}')
print(f'ASCII Printable Characters: {string.printable}')
```

The first line of output is the letters of the alphabet, both uppercase and lowercase. The second line of output is the first line, numbers 0 to 9, plus the characters:

```
!"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~
```

challenge

Try this variation:

Taking this one step further, we can also see the underlying code points of the characters, by converting all non-ASCII characters to their two- or four-digit Unicode syntaxes: `\uNN` or `\uNNNN`. This is done with the `encode('unicode_escape')` method.

Enter the below code, then select the TRY IT button to run the file.

```
with open(path_1, encoding = 'latin2') as text_latin2:
    print('\nPlain Text from path_1 using Latin-2 encoding
          with code points:')
    for line in text_latin2:
        line = line.strip()
        print(line.encode('unicode_escape'))
```

Your output should display as shown below.

```
Plain Text from path_1 using Latin-2 encoding with code
points:
b'Asynchrones Schreiben erh\\u0102\\u015bht die Leistung ein
wenig, es verringert jedoch die Zuverl\\u0102\\xa4ssigkeit,
da die Gefahr eines Datenverlusts besteht, falls der Server
zwischen der Annahmebest\\u0102\\xa4tigung des
Schreibauftrags und dem tats\\u0102\\xa4chlichen
Schreibvorgang auf der Festplatte abst\\u0102\\u017arzt.'
b'Aus diesem Grund erhalten Sie f\\u0102\\u017ar jede bei
uns get\\u0102\\xa4tigte Bestellung eine
Annahmebest\\u0102\\xa4tigung, die noch einmal alle
wichtigen Daten des Auftrags zusammenfa\\u0102\\x9ft, und
auf der unsere Notfall Hotline angegeben ist, unter der Sie
uns im Falle von auftretenden Schwierigkeiten am
Leistungstag erreichen k\\u0102\\u015bnnen.'
b'Durch Vorlage der Annahmebest\\u0102\\xa4tigung Ihrer
Fakult\\u0102\\xa4t und des ausgef\\u0102\\u017allten
Antrags auf Registrierung im Studierendensekretariat
(deutsche Promovierende) oder beim International Admissions
und Services (IAS) (internationale Promovierende) werden Sie
als Doktorand*in registriert.'
```

Use UTF-8 Encoding

Now, let's read the same file but with UTF-8 encoding. Notice, we did not need to specify the encoding type as UTF-8 is the default. Predict what you think will happen, enter the below code, then select the TRY IT button to run the file.

```
with open(path_1, encoding = 'utf8') as text_utf8:
    print("\nPlain Text from path_1 using UTF-8 encoding:")
    for line in text_utf8:
        line = line.strip()
        print(line)
```

Your output should display as shown below.

```
Plain Text from path_1 using UTF-8 encoding:
Asynchrones Schreiben erhöht die Leistung ein wenig, es
verringert jedoch die Zuverlässigkeit, da die Gefahr eines
Datenverlusts besteht, falls der Server zwischen der
Annahmebestätigung des Schreibauftrags und dem tatsächlichen
Schreibvorgang auf der Festplatte abstürzt.
Aus diesem Grund erhalten Sie für jede bei uns getätigte
Bestellung eine Annahmebestätigung, die noch einmal alle
wichtigen Daten des Auftrags zusammenfaßt, und auf der unsere
Notfall Hotline angegeben ist, unter der Sie uns im Falle von
auftretenden Schwierigkeiten am Leistungstag erreichen können.
Durch Vorlage der Annahmebestätigung Ihrer Fakultät und des
ausgefüllten Antrags auf Registrierung im
Studierendensekretariat (deutsche Promovierende) oder beim
International Admissions und Services (IAS) (internationale
Promovierende) werden Sie als Doktorand*in registriert.
```

Note, the word 'erhöht' displayed in the first line of the file's text is now stored completely as 'erhöht' because we used UTF-8 encoding, rather than Latin-2.

challenge

Try this variation:

We can also use regular expressions along with unicode to find matches of specified patterns in the text.

In the code shown below, we use the following regex pattern:

```
re.search(r"\\u015b\\w*", str(line_1)).
```

- We use the `readlines` method to store a list of all lines, where each item in the list represents its own line. This allows us to easily pull the values from line 1 of the file.
- `\\u015b\\w*` represents the pattern we are looking for in the string version of `line_1`. We are looking for 0 or more instances where a word matches the pattern.
- Using the `group` method, we can print out the entire word that the match is contained in.

Enter the below code, then select the TRY IT button to run the file.

```
import re

lines_in_text = open(path_1, encoding =
    'latin2').readlines()
line_1 = lines_in_text[0]
line_1 = line_1.encode('unicode_escape')
print("\nLine 1 of the Text:", str(line_1))

unicode_in_line_1 = re.search(r"\\u015b\\w*", str(line_1))
print("Words with Unicode 'u015b':",
    unicode_in_line_1.group())
```

Your output should display as shown below.

```
Line 1 of the Text: b'Asynchrones Schreiben
erh\\u0102\\u015bht die Leistung ein wenig, es verringert
jedoch die Zuverl\\u0102\\xa4ssigkeit, da die Gefahr eines
Datenverlusts besteht, falls der Server zwischen der
Annahmebest\\u0102\\xa4tigung des Schreibauftrags und dem
tats\\u0102\\xa4chlichen Schreibvorgang auf der Festplatte
abst\\u0102\\u017arzt.\\n'
Words with Unicode 'u015b': \\u015bht
```

Chunk & Chink Text

Definitions: Chunking & Chinking

We are ready to start parsing text into groups following certain criteria. This is commonly referred to as chunking and chinking.

- **Chunking**, also known as *chunk parsing*, allows the computer to recognize phrases, which could include an individual word or a grouping of words referred to as a *chunk*.
 - Whether it be in the form of one word or a grouping of words, each phrase represents its own grammatical element.
- **Chinking**, often used with chunking, is essentially a way for us to *unchunk* or break up chunked phrases already identified. The *chunk*, or the identified pattern, which is removed is called the *chink*.

▼ Chunk patterns

For your reference and as a reminder, listed below is a comprehensive list of all tags and their defined tagged parts of speech.

Tag	Part of Speech
CC	conjunction, coordinating
CD	numeral, cardinal
DT	determiner
EX	existential there
FW	foreign word
IN	preposition or conjunction, subordinating
JJ	adjective or numeral, ordinal
JJR	adjective, comparative
JJS	adjective, superlative
LS	list item marker
MD	modal auxiliary
NN	noun, common, singular or mass
NNP	noun, proper, singular
NNPS	noun, proper, plural
NNS	noun, common, plural
PDT	pre-determiner
POS	genitive marker
PRP	pronoun, personal

PRP\$	pronoun, possessive
RB	adverb
RBR	adverb, comparative
RBS	adverb, superlative
RP	particle
SYM	symbol
TO	“to” as preposition or infinitive marker
UH	interjection
VB	verb, base form
VBD	verb, past tense
VP	verb, present participle or gerund
VRB	verb, past participle
VBP	verb, present tense, not 3rd person singular
VBZ	verb, present tense, 3rd person singular
WDT	WH-determiner
WP	WH-pronoun
WP\$	WH-pronoun, possessive
WRB	Wh-adverb

Listed in the table below are symbols we use when chunking and their usages.

Symbol	Use
<>	The criteria go in between these angle brackets.
*	The criteria can occur 0 or more times.
+	The criteria must occur at least once.
?	The criteria can optionally occur once or not at all.
.	The criteria cannot be on a new line.

Note that POS tags are enclosed between angular brackets (<>), which are enclosed between curly brackets ({}).

Chunk Text

Import the applicable functions from the NLTK library used for chunking and chunking in these exercises.

```
from nltk.tokenize import word_tokenize
from nltk import pos_tag, RegexpParser
```

For this exercise, we will analyze the string contained in the variable called `head_quote`.

```
head_quote = "It's strange how a word, a phrase, a sentence, can  
feel like a blow to the head."
```

As usual, we need to tokenize the string by word, then find the part of speech of each before we can start chunking phrases in the text.

```
head_quote_words = word_tokenize(head_quote)  
head_quote_tags = pos_tag(head_quote_words)
```

Now, to begin chunking, we need to define a *chunk grammar*.

- A chunk grammar is a set of rules on how sentences should be chunked, which very often uses regex.

In the below example, we will chunk noun phrases (phrases built around a noun), which is what NP stands for.

- This command moves each word into its own tuple in order to store each word and each word's part of speech tag in a single variable.

Then, we create a chunk parser to apply the rules from this chunk grammar using `RegexpParser` which we imported earlier from the `nltk` module. Lastly, we use the `parse` method to apply the chunk to the text.

▼ What are noun phrases?

Noun phrases consist of a noun and a modifier (modifiers function as adjectives or adverbs by aiding the intent of the speech to be more specific).

Each of the below examples illustrates how a modifier (e.g., "older") is used next to a noun (e.g., "brother") to together become a *noun phrase*.

Examples:

- The **devil in disguise**.
- The **spotted dog**.
- A **red chair** was put in the room.
- My **older brother** loves to eat.

Enter the below code, then select the TRY IT button to run the file.

```
chunk_grammar_NP = r"NP: {<DT>?<JJ>*<NN>}"  
chunk_parser_NP = RegexpParser(chunk_grammar_NP)  
head_quote_chunked = chunk_parser_NP.parse(head_quote_tags)  
print("\nChunked Noun Phrases in 'head_quote': \n",  
      head_quote_chunked)
```


▼ Explained: chunk_grammar

```
chunk_grammar_NP = r"NP: {<DT>?<JJ>*<NN>}"
```

In the above code, we create a variable called `chunk_grammar` which contains the grammar rules / pattern we are looking for.

- NP: We tell the computer to check for noun phrases, which we call NP. Code written in the brackets following "NP: defines noun phrases.

- <DT>?: If the computer finds an optional determiner (? means optional), then it will pass and move to the next criteria.

- <JJ>*: This means that the noun phrase can have any number of adjectives (* means *any number*).

- <NN>: This means that the noun phrase must have a noun following any number of adjectives.

Your output should display as shown below.

Chunked Noun Phrases in 'head_quote':

```
(S
  It/PRP
  's/VBZ
  strange/JJ
  how/WRB
  (NP a/DT word/NN)
  ,/,
  (NP a/DT phrase/NN)
  ,/,
  (NP a/DT sentence/NN)
  ,/,
  can/MD
  feel/VB
  like/IN
  (NP a/DT blow/NN)
  to/TO
  (NP the/DT head/NN)
  ./.)
```

▼ What does the output mean?

The “S” at the beginning of the output represents the highest level in the hierarchy, the “sentence”.

Word(s)	Chunked Group	NP? (Y/N)
It	Personal Pronoun	No
's	Present Tense Verb	No

strange	Adjective	No
how	WH Adverb	No
a word	“a”=Determiner, “word”=Noun	Yes
,	Comma	No
a phrase	“a”=Determiner, “phrase”=Noun	Yes
,	Comma	No
a sentence	“a”=Determiner, “sentence”=Noun	Yes
,	Comma	No
can	Modal Auxiliary	No
feel	Verb	No
like	Subordinate Preposition	No
a blow	“a”=Determiner, “blow”=Noun	Yes
to	“to” Marker	No
the head	“the”=Determiner, “head”=Noun	Yes
.	Period Mark	No

challenge

Try this variation:

In this variation, we will chunk verb phrases (phrases built around a verb), which is what VP stands for.

▼ What are verb phrases?

Verb phrases function as verbs in a sentence. They consist of a verb and a verb modifier, which aid the intent of the speech to be more specific.

Each of the below examples illustrates how a modifier (e.g., “might”) is used next to a verb (e.g., “go”) to together become a *verb phrase*.

Examples

- I **might go** to the movies later.
- You **shouldn’t do** that.
- **Are you trying** to text too much?
- My older brother **loves eating**.

Enter the below code, then select the TRY IT button to run the file.

```
painting = "I finally told my mother my passion, which is  
that I love painting."  
painting_words = word_tokenize(painting)  
painting_tags = pos_tag(painting_words)  
  
chunk_grammar_VP = "VP: {<VPD>*<VBG>}"  
chunk_parser_VP = RegexpParser(chunk_grammar_VP)  
print("\nChunked Verb Phrases in 'painting': \n",  
      chunk_parser_VP.parse(painting_tags))
```

Your output should display as shown below.

```
Chunked Verb Phrases in 'painting':  
(S  
  I/PRP  
  finally/RB  
  told/VBD  
  my/PRP$  
  mother/NN  
  my/PRP$  
  passion/NN  
  ,/,  
  which/WD  
  is/VBZ  
  that/IN  
  I/PRP  
  love/VBP  
  (VP painting/VBG)  
  ./.)
```

Chink Text

If chunking is creating a meaningful group of text, chinking is removing a subset of text from the group. In this next exercise, we will work with the string variable called `medal_quote`.

```
medal_quote = "These boys collected the bronze medal for third  
team place and were only ten points away from earning a  
gold medal."
```

Moving forward, we will be *chunking* this data, so we can run through an example of *chinking* the chunked data. Before we can begin chunking, we need to tokenize each word, then find its each part of speech.

```
medal_quote_tokens = word_tokenize(medal_quote)
medal_quote_tags = pos_tag(medal_quote_tokens)
```

Now, to begin chunking, we need to define a *chunk grammar*. We want to chunk everything in the string except for adjectives. The process of removing adjectives from the chunks is chunking.

- {<.*>+} will chunk everything.

- }<JJ>{ will exclude chunking of adjectives. **(This is chunking.)**

We create a chunk parser with the rules from the chunk grammar. We, then, use parse to chunk and/or chunk the text.

Enter the below code, then select the TRY IT button to run the file.

```
chunk_grammar_noJJ = """NoAdjChunk: {<.*>+}
                        }<JJ>{"""
chunk_parser_noJJ = RegexpParser(chunk_grammar_noJJ)
medal_quote_chunked = chunk_parser_noJJ.parse(medal_quote_tags)
print("\nChunked 'medal_quote' without Adjectives: \n",
      medal_quote_chunked)
```

Your output should display as shown below. You can see the chunks which start with (NoAdjChunk. The adjectives (/JJ) stand alone.

Chunked 'medal_quote' without Adjectives:

```
(S
  (NoAdjChunk
    These/DT
    boys/NNS
    collected/VBD
    the/DT
    bronze/NN
    medal/NN
    for/IN)
  third/JJ
  (NoAdjChunk
    team/NN
    place/NN
    and/CC
    were/VBD
    only/RB)
  ten/JJ
  (NoAdjChunk
    points/NNS
    away/RB
    from/IN
    earning/VBG
    a/DT
    gold/NN
    medal/NN
    ./.)
```

challenge

Try this variation:

In the code below, we chunk noun phrases and chunk verbs from the variable `head_quote`.

Enter the below code, then select the TRY IT button to run the file.

```

head_quote = "It's strange how a word, a phrase, a sentence,
              can feel like a blow to the head."
head_quote_words = word_tokenize(head_quote)
head_quote_tags = pos_tag(head_quote_words)

chunk_grammar_NP_noVB = """
    NP: {<DT><NN.*><.*>*<NN.*>}
    }<VB.*>{""

chunk_parser_NP_noVB = RegexpParser(chunk_grammar_NP_noVB)
head_quote_chunked2 =
    chunk_parser_NP_noVB.parse(head_quote_tags)
print("\nChunked Noun Phrases in 'head_quote' without Verbs:
      \n", head_quote_chunked2)

```

Your output should display as shown below.

```

Chunked Noun Phrases in 'head_quote' without Verbs:
(S
  It/PRP
  's/VBZ
  strange/JJ
  how/WRB
  (NP
    a/DT
    word/NN
    ,/,
    a/DT
    phrase/NN
    ,/,
    a/DT
    sentence/NN
    ,/,
    can/MD)
  feel/VB
  (NP
    like/IN
    a/DT
    blow/NN
    to/TO
    the/DT
    head/NN)
  ./.)

```

Use NER

Definition: NER

Another way to parse text is by entity.

- Named Entity Recognition, or *NER* for short, is the method computer applications use to identify words or phrases as entities.
 - For example, *Codio* would be identified as an entity in the sentence: “*Open the Codio website.*”
- An entity is a noun that exists uniquely and autonomously.

▼ Using NER vs. tagging POS

- **Tagging POS** allows us to identify the grammar group the word belongs to (e.g., nouns, verbs, adjectives).
- **NER** uses the output from tagged POS in order to check whether each identified noun is a named entity (e.g., person, location, organization).

Use ne_chunk

Import the applicable functions from the NLTK library used for using NER in these exercises.

```
from nltk.tokenize import word_tokenize
from nltk import pos_tag, ne_chunk
```

For this exercise, we will analyze the string contained in the variable called `coffee`.

```
coffee = "Angela Merkel grabbed the cup of coffee and took a sip
as she stepped around the corner of Starbucks."
```

First, we need to tokenize the string by word before we can start, then find its tag.

```
coffee_tokens = word_tokenize(coffee)
coffee_tags = pos_tag(coffee_tokens)
```

In the code below, we use the `ne_chunk` method that we imported earlier from the `nltk` module to chunk NERs from the text.

Enter the below code, then select the TRY IT button to run the file.

```
coffee_NEs = ne_chunk(coffee_tags)
print(f"\nName Entities in 'coffee': {coffee_NEs}")
```

Your output should display as shown below.

```
Name Entities in 'coffee': (S
  (PERSON Angela/NNP)
  (PERSON Merkel/NNP)
  grabbed/VBD
  the/DT
  cup/NN
  of/IN
  coffee/NN
  and/CC
  took/VBD
  a/DT
  sip/NN
  as/IN
  she/PRP
  stepped/VBD
  around/IN
  the/DT
  corner/NN
  of/IN
  (GPE Starbucks/NNP)
  ./.)
```

challenge

Try this variation:

Create a user-defined function to extract name entities from the text. Write the code to produce the following expected output, then select the TRY IT button to run the file and check your work. The expected output is shown below.


```
Name Entities in 'coffee' using User-Defined Function: (S
(PERSON Angela/NNP)
(PERSON Merkel/NNP)
grabbed/VBD
the/DT
cup/NN
of/IN
coffee/NN
and/CC
took/VBD
a/DT
sip/NN
as/IN
she/PRP
stepped/VBD
around/IN
the/DT
corner/NN
of/IN
(GPE Starbucks/NNP)
./.)
```

▼ Solution

One possible solution is shown below, where we can create function `find_NEs` that applies `ne_chunk` on a list of all tokens tagged with their parts of speech to find the named entities in the specified text.

```
def find_NEs(text):
    tokens = word_tokenize(text)
    tags = pos_tag(tokens)
    return(ne_chunk(tags))

print(f"\nName Entities in 'coffee' using User-Defined
      Function: {find_NEs(coffee)}")
```

Use `hasattr`

In this next exercise, we will continue to work with the variable space. The `hasattr` function takes an object and an attribute. If the object has the given attribute, the function returns `True`. If not, it returns `False`. We want all words with the `'label'` attribute with the value `'NE'` to be returned by the `find_NEs_pretty2` function.

Enter the below code, then select the TRY IT button to run the file.

```
def find_NEs_pretty2(text):
    tokens = word_tokenize(text)
    tags = pos_tag(tokens)
    NEs = ne_chunk(tags, binary=True)
    entities = []
    for chunked_word in NEs:
        if hasattr(chunked_word, 'label') and chunked_word.label()
           == 'NE':
            named_entity = ' '.join(word[0] for word in chunked_word)
            entities.append(named_entity)
    return entities

space = 'Many try to replicate the magic of Star Trek, but none
        have succeeded.'

print(f"\nName Entities in 'space' using hasattr():
      {find_NEs_pretty2(space)}")
```

Your output should display as shown below.

```
Name Entities in 'space' using hasattr(): ['Star Trek']
```

challenge

Try this variation:

Let's see what happens if we change the binary value from True to False (the default value).

Predict what you think will happen, enter the below code, then select the TRY IT button to run the file.

```
def find_NEs_pretty2(text):
    tokens = word_tokenize(text)
    tags = pos_tag(tokens)
    NEs = ne_chunk(tags, binary=False)
    entities = []
    for chunked_word in NEs:
        if hasattr(chunked_word, 'label') and
            chunked_word.label() == 'NE':
            named_entity = ' '.join(word[0] for word in
                chunked_word)
            entities.append(named_entity)
    return entities

space = 'Many try to replicate the magic of Star Trek, but
none have succeeded.'

print(f"\nName Entities in 'space' using hasattr():
{find_NEs_pretty2(space)}")
```

Your output should display as shown below.

```
Name Entities in 'space' using hasattr(): []
```

- When we set binary equal to True, the named entities will be tagged as NE.
- When binary is set to False (the default value), named entity category labels will be used, such as PERSON.

Segment Text

Definition: Segmentation

We have seen how we can divide a text into tokens. Another way in which we can parse a text is through segmentation.

- With *tokenization*, text is broken into smaller chunks of text, either by word or by sentence using the `nltk.tokenize` package.
- With *segmentation*, text is simply divided into semantically-oriented continuous segments. Tokenization fits into this broader definition of segmentation.
- There are several different tokenizers which can be used in the NLTK library. When we refer to *segmentation*, it is often to point out the problem of not having a one-shoe-fits-all solution.

Use RegexpTokenizer

Using the `nltk.tokenize` module, we can import the `RegexpTokenizer` function from the NLTK library.

```
from nltk.tokenize import RegexpTokenizer
```

The imported `RegexpTokenizer` function allows us to break this text by a pattern determined with a regular expression, where each word is grouped in its own string within a list.

▼ Important - regex groups

Regex Group	Group Definition
[a-z]	lowercase alphabet values ('zyxabc')
[A-Za-z]+	upper- and lowercase values ('ZyxAbc')
[A-Za-z-.,]+	upper- and lowercase, and - or . values ('Well-deserved')
	space values (' ')
	no space values ('nospaces')
+	space and comma values (',')
digit values (123987)	
[0-9]	numbers 0-9 values (1)
	word values ('dog', 'mom')
	no word values ('&%')
tab character values (')	
	new line character values ('')

We want to segment a string based on spaces between words. Start by creating a pattern with `RegexpTokenizer`. The `gaps = True` means that Python uses the pattern to identify gaps for tokenization. Use the `tokenize()` method on the pattern and pass a string to the tokenizer. The end result is a list of words based on the original pattern.

Enter the code below, then select the TRY IT button to run the file.

```
white_space_pattern = RegexpTokenizer('\s+', gaps = True)

codio = 'Codio is the best computer science learning tool.'
codio_tokenized = white_space_pattern.tokenize(codio)
print(f"\nOriginal Output of 'codio': {codio}")
print(f"\nRegexp Tokenizer Tokenized Output of 'codio':
      {codio_tokenized}")
```

Your output should display as shown below:

```
Original Output of 'codio': Codio is the best computer science
learning tool.
Regexp Tokenizer Tokenized Output of 'codio': ['Codio', 'is',
'the', 'best', 'computer', 'science', 'learning', 'tool.']
```

▼ **Did you notice?**

```
Regexp Tokenizer Tokenized Output of 'codio': ['Codio', 'is',
'the', 'best', 'computer', 'science', 'learning', 'tool.']
```

The RegexpTokenizer method does not recognize punctuation as their own tokens.

challenge

Try this variation:

Assume the following string:

```
magic = 'My teacher, Ms. Sanford, said there is always magic  
in the air if you believe in it.'
```

Write a script that creates a list of uppercase tokens from the string `magic`. Then select the TRY IT button to run the file and check your work.

The expected output is shown below.

```
Regex Tokenizer by Uppercase Tokenized Output of 'magic':  
['M', 'M', 'S']
```

▼ Solution

One possible solution is shown below, where we can use `RegexTokenizer` to tokenize by uppercase letter.

```
uppercase_pattern = RegexTokenizer('[A-Z]+', gaps = False)  
magic = 'My teacher, Ms. Sanford, said there is always magic  
in the air if you believe in it.'
```

```
magic_tokenized = uppercase_pattern.tokenize(magic)  
print(f"\nRegex Tokenizer by Uppercase Tokenized Output of  
'magic': {magic_tokenized}")
```

Notice that `gaps` is set to `False`. If not, Python would return a list of three strings. Segmentation would happen at the capital letters. Instead, we want just the capital letters, so `gaps` is set to `False`.

Manually Segment Text

We can also manually segment text by bit values. In the code shown below, we create a function called `segment_text`, which takes in two arguments: `text` and `segment`. `text` represents a string that needs to be segmented. `segment` is a pattern consisting of 1's and 0's. A 0 indicates that character remains unchanged. A 1 means splitting the original string at this character.

- We create an empty list called `words` to be later used for storing all segmented words from the `text`.
- We create a counter variable called `last`, which is meant to represent the last index value we stopped on.
- We use a `for` loop to loop through the range of the length of the provided `segment`.
- Within the `for` loop, we use a conditional statement to declare that if an element in the provided `segment` is equal to the value of '1', then we will append a slice of the `text` from the index value equal to the variable `last`'s value (which starts on 0) to the index value of equal to the element's index value plus 1. Then, we re-assign the value of the variable `last` to be equal to the element's index value plus 1.
- Outside of the `for` loop, we append the element value from the provided `text` to the list called `words`.
- When the function is finished, we return the list called `words`.

```
def segment_text(text, segment):
    words = []
    last = 0
    for element in range(len(segment)):
        if segment[element] == '1':
            words.append(text[last: element + 1])
            last = element + 1
    words.append(text[last:])
    return words
```

Enter the code below, then select the TRY IT button to run the file.

```
happy_text = 'Happydaysarealwayshere.'
seg_happy_text = '00001000100100000100010'
print(f"\nOriginal Output of 'happy_text': {happy_text}")
print(f"\nManually Segmented Output of 'happy_text': {segment_text(happy_text, seg_happy_text)}")
```

Your output should display as shown below:

```
Original Output of 'happy_text': Happydaysarealwayshere.
Manually Segmented Output of 'happy_text': ['Happy', 'days', 'are', 'always', 'here', '.']
```

challenge

Try this variation:

Using string `hotel_ad`, write the code to produce the following expected output using the function `segment_text` that we created earlier. Then select the TRY IT button to run the file and check your work.

```
hotel_ad = 'Theroomyouneedattheresortyoutrustwithwell-appointedroomsandfriendly,personalizedservice.'
```

The expected output is shown below.

```
Manually Segmented Output of 'hotel_ad': ['The', 'room', 'you', 'need', 'at', 'the', 're', 'you', 'trust', 'with', 'well-appointed', 'rooms', 'and', 'friendly', ',', 'personalized', 'service', '.']
```

▼ Solution

One possible solution is shown below, where we can manually segment `hotel_ad` using function `segment_text`.

```
seg_hotel_ad = '0010001001000101001000001001000010001000000000000100010010000000110000000000'
print(f"\nManually Segmented Output of 'hotel_ad': {segment_text(hotel_ad, seg_hotel_ad)}")
```